2、中级面试题

由 涤生老师-健哥创建, 最后修改于一月 04, 2025

- 1、为什么需要消息系统，mysql 不能满足需求吗？（了解）
- 2、Kafka目前有哪些内部topic，它们都有什么特征？各自的作用又是什么？（必背）
- 3、优先副本是什么？它有什么特殊的作用？（理解）
- 4、Kafka有哪几处地方有分区分配的概念？简述大致的过程及原理（理解）
- 5、简述Kafka的日志目录结构（理解背诵）
- 6、Kafka中有那些索引文件？（理解）
- 7、如果我指定了一个offset，Kafka怎么查找到对应的消息？（了解）
- 8、如果我指定了一个timestamp，Kafka怎么查找到对应的消息？（了解）
- 9、聊一聊你对Kafka的Log Retention的理解（理解背诵）
- 10、聊一聊你对Kafka的Log Compaction（日志压缩）的理解（理解）
- 11、聊一聊你对Kafka底层存储的理解（理解）
  - i、页缓存
  - ii、零拷贝
- 12、聊一聊Kafka的延时操作的原理（了解）
- 13、聊一聊Kafka控制器的作用（了解）
- 14、Kafka的旧版Scala的消费者客户端的设计有什么缺陷？（了解）
- 15、消费再均衡有哪些场景及相应的原理是什么？（理解）
  - 第一阶段（FIND_COORDINATOR）
  - 第二阶段（JOIN_GROUP）
  - 第三阶段（SYNC_GROUP）
  - 第四阶段（HEARTBEAT）
- 16、Kafka中的生产幂等是怎么实现的？（理解）
- 17、Kafka支持什么语义，怎么实现Exactly Once？（理解背诵）
- 18、Kafka producer的写入数据过程？（背诵）
- 19、Kafka读取消息是推还是拉的模式？有什么好处？（了解）
- 20、如何调整生产环境中Kafka主题的分区数量？（理解）
- 21、如何重平衡Kafka集群？（了解）
- 22、如何查看消费者组是否存在滞后消费？（了解）

1、为什么需要消息系统，mysql 不能满足需求吗？（了解）

（1）解耦：

允许你独立的扩展或修改两边的处理过程，只要确保它们遵守同样的接口约束。

（2）冗余：

消息队列把数据进行持久化直到它们已经被完全处理，通过这一方式规避了数据丢失风险。许多消息队列所采用的”插入-获取-删除”范式中，在把一个消息从队列中删除之前，需要你的处理系统明确的指出该消息已经被处理完毕，从而确保你的数据被安全的保存直到你使用完毕。

（3）扩展性：

因为消息队列解耦了你的处理过程，所以增大消息入队和处理的频率是很容易的，只要另外增加处理过程即可。

（4）灵活性 & 峰值处理能力：

在访问量剧增的情况下，应用仍然需要继续发挥作用，但是这样的突发流量并不常见。如果为以能处理这类峰值访问为标准来投入资源随时待命无疑是巨大的浪费。使用消息队列能够使关键组件顶住突发的访问压力，而不会因为突发的超负荷的请求而完全崩溃。

（5）可恢复性：

系统的一部分组件失效时，不会影响到整个系统。消息队列降低了进程间的耦合度，所以即使一个处理消息的进程挂掉，加入队列中的消息仍然可以在系统恢复后被处理。

（6）顺序保证：

在大多使用场景下，数据处理的顺序都很重要。大部分消息队列本来就是排序的，并且能保证数据会按照特定的顺序来处理。（Kafka 保证一个 Partition 内的消息的有序性）

（7）缓冲：

有助于控制和优化数据流经过系统的速度，解决生产消息和消费消息的处理速度不一致的情况。

（8）异步通信：

很多时候，用户不想也不需要立即处理消息。消息队列提供了异步处理机制，允许用户把一个消息放入队列，但并不立即处理它。想向队列中放入多少消息就放多少，然后在需要的时候再去处理它们。

2、Kafka目前有哪些内部topic，它们都有什么特征？各自的作用又是什么？（必背）

__consumer_offsets：作用是保存 Kafka 消费者的位移信息
__transaction_state：用来存储事务日志消息

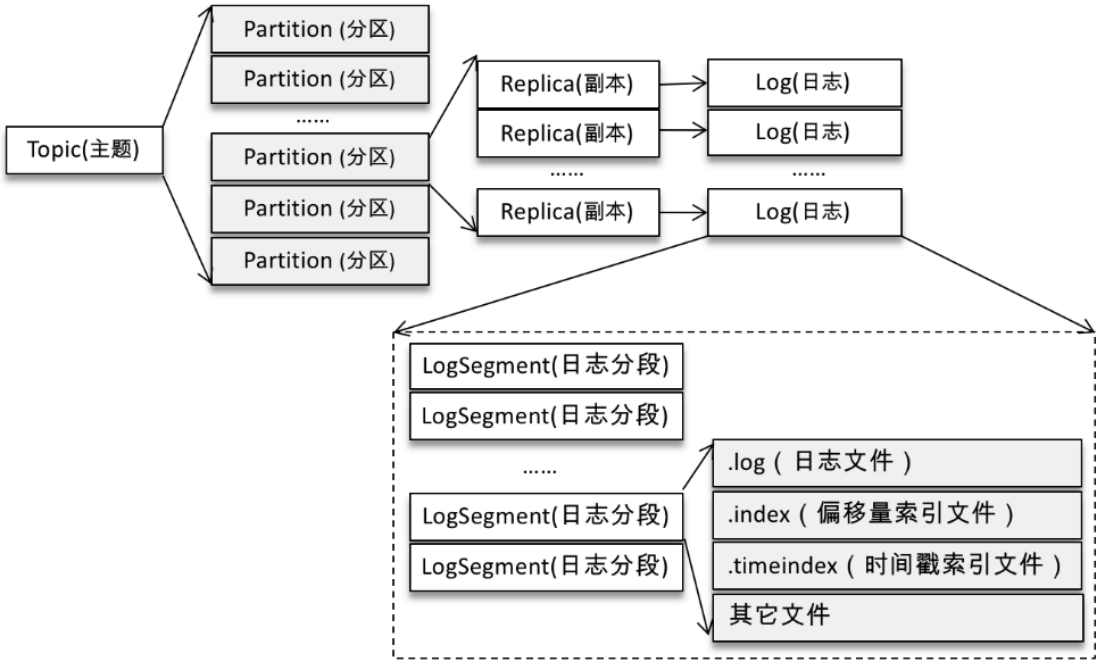
3、优先副本是什么？它有什么特殊的作用？（理解）

所谓的**优先副本是指在AR集合列表中的第一个副本。**
理想情况下，**优先副本就是该分区的leader 副本，所以也可以称之为 preferred leader**。Kafka 要确保所有topic的优先副本在 Kafka 集群中均匀分布，这样就保证了所有分区的 leader 均衡分布。以此来促进集群的负载均衡，这一行为也可以称为“分区平衡”。

4、Kafka有哪几处地方有分区分配的概念？简述大致的过程及原理（理解）

1. **生产者的分区分配**是指为每条消息指定其所要发往的分区。可以编写一个具体的类实现org.apache.kafka.clients.producer.Partitioner接口。
2. **消费者中的分区分配**是指为消费者指定其可以消费消息的分区。Kafka 提供了消费者客户端参数 partition.assignment.strategy 来设置消费者与订阅topic之间的分区分配策略。
3. **分区副本的分配**是指为集群制定创建topic时的分区副本分配方案，即在哪个 broker 中创建哪些分区的副本。kafka-topics.sh 脚本中提供了一个 replica-assignment 参数来手动指定分区副本的分配方案。

5、简述Kafka的日志目录结构（理解背诵）



Kafka 中的消息是以topic为基本单位进行归类的，各个topic在逻辑上相互独立。**每个topic又可以分为一个或多个分区**。不考虑多副本的情况，**一个分区对应一个日志（Log）**。为了防止 Log 过大，Kafka 又引入了日志分段（LogSegment）的概念，将 Log 切分为多个 LogSegment，相当于一个巨型文件被平均分配为多个相对较小的文件。

Log 和 LogSegment 也不是纯粹物理意义上的概念，**Log 在物理上只以文件夹的形式存储，而每个 LogSegment 对应于磁盘上的一个日志文件和两个索引文件，以及可能的其他文件（比如以“.txnindex”为后缀的事务索引文件）**

6、Kafka中有那些索引文件？（理解）

每个日志分段文件对应了**两个索引文件**，主要用来提高查找消息的效率。
偏移量索引文件用来建立消息偏移量（offset）到物理地址之间的映射关系，方便快速定位消息所在的物理文件位置
时间戳索引文件则根据指定的时间戳（timestamp）来查找对应的偏移量信息。

7、如果我指定了一个offset，Kafka怎么查找到对应的消息？（了解）

Kafka是通过seek() 方法来指定消费的，在执行seek() 方法之前要去执行一次poll()方法，等到分配到分区之后会去对应的分区的指定位置开始消费，如果指定的位置发生了越界，那么会根据auto.offset.reset 参数设置的情况进行消费。

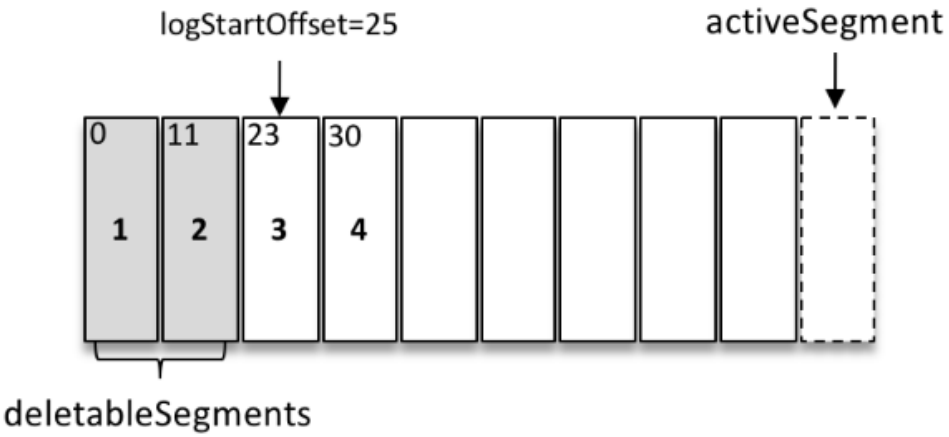
8、如果我指定了一个timestamp，Kafka怎么查找到对应的消息？（了解）

Kafka提供了一个 offsetsForTimes() 方法，通过 timestamp 来查询与此对应的分区位置。offsetsForTimes() 方法的参数 timestampsToSearch 是一个 Map 类型，key 为待查询的分区，而 value 为待查询的时间戳，**该方法会返回时间戳大于等于待查询时间的第一条消息对应的位置和时间戳**，对应于 OffsetAndTimestamp 中的 offset 和 timestamp 字段。

9、聊一聊你对Kafka的Log Retention的理解（理解背诵）

日志删除（Log Retention）：按照一定的保留策略直接删除不符合条件的日志分段。我们可以通过 broker 端参数 **log.cleanup.policy** 来设置日志清理策略，此参数的默认值为“delete”，即采用日志删除的清理策略。

1. **基于时间**
   日志删除任务会检查当前日志文件中是否有保留时间超过设定的阈值（retentionMs）来寻找可删除的日志分段文件集合（deletableSegments）retentionMs 可以通过 broker 端参数 log.retention.hours、log.retention.minutes 和 log.retention.ms 来配置，其中 log.retention.ms 的优先级最高，log.retention.minutes 次之，log.retention.hours 最低。默认情况下只配置了 log.retention.hours 参数，其值为168，故默认情况下日志分段文件的保留时间为7天。
   删除日志分段时，首先会从 Log 对象中所维护日志分段的跳跃表中移除待删除的日志分段，以保证没有线程对这些日志分段进行读取操作。然后将日志分段所对应的所有文件添加上“.deleted”的后缀（当然也包括对应的索引文件）。最后交由一个以“delete-file”命名的延迟任务来删除这些以“.deleted”为后缀的文件，这个任务的延迟执行时间可以通过 file.delete.delay.ms 参数来调配，此参数的默认值为60000，即1分钟。
2. **基于日志大小**
   日志删除任务会检查当前日志的大小是否超过设定的阈值（retentionSize）来寻找可删除的日志分段的文件集合（deletableSegments）。retentionSize 可以通过 broker 端参数 log.retention.bytes 来配置，默认值为-1，表示无穷大。**注意 log.retention.bytes 配置的是 Log 中所有日志文件的总大小**，而不是单个日志分段（确切地说应该为 .log 日志文件）的大小。**单个日志分段的大小由 broker 端参数 log.segment.bytes 来限制**，默认值为1073741824，即 1GB。
   这个删除操作和基于时间的保留策略的删除操作相同。
3. **基于日志起始偏移量**
   基于日志起始偏移量的保留策略的判断依据是某日志分段的下一个日志分段的起始偏移量 baseOffset 是否小于等于 logStartOffset，若是，则可以删除此日志分段。

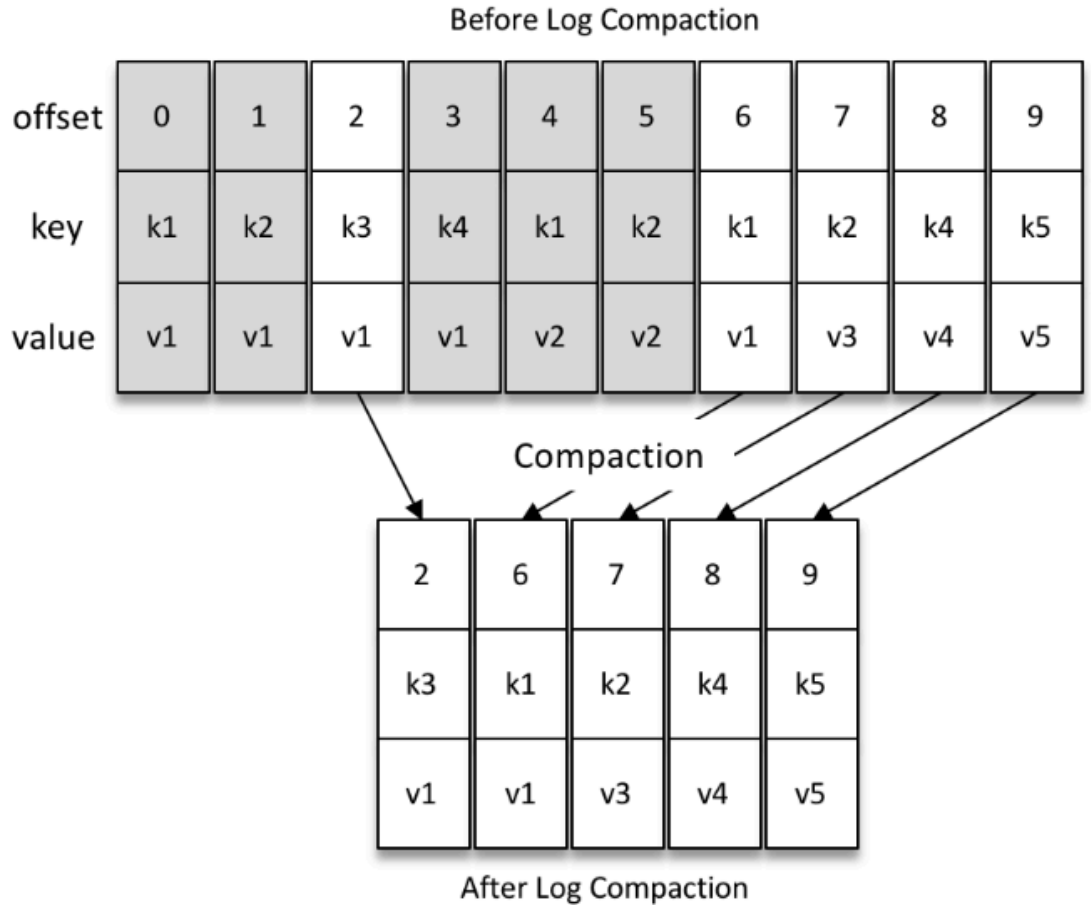


如上图所示，假设 logStartOffset 等于25，日志分段1的起始偏移量为0，日志分段2的起始偏移量为11，日志分段3的起始偏移量为23，通过如下动作收集可删除的日志分段的文件集合 deletableSegments：

从头开始遍历每个日志分段，日志分段1的下一个日志分段的起始偏移量为11，小于 logStartOffset 的大小，将日志分段1加入 deletableSegments。
日志分段2的下一个日志偏移量的起始偏移量为23，也小于 logStartOffset 的大小，将日志分段2加入 deletableSegments。
日志分段3的下一个日志偏移量在 logStartOffset 的右侧，故从日志分段3开始的所有日志分段都不会加入 deletableSegments。
收集完可删除的日志分段的文件集合之后的删除操作同基于日志大小的保留策略和基于时间的保留策略相同

10、聊一聊你对Kafka的Log Compaction（日志压缩）的理解（理解）

针对每个消息的 key 进行整合，**对于有相同 key 的不同 value 值，只保留最后一个版本。**如果要采用日志压缩的清理策略，就需要将 log.cleanup.policy 设置为“compact”，并且还需要将 log.cleaner.enable （默认值为 true）设定为 true。



如上图所示，Log Compaction 对于有相同 key 的不同 value 值，只保留最后一个版本。**如果应用只关心 key 对应的最新 value 值，则可以开启 Kafka 的日志清理功能，Kafka 会定期将相同 key 的消息进行合并，只保留最新的 value 值。**

11、聊一聊你对Kafka底层存储的理解（理解）

i、页缓存

页缓存是操作系统实现的一种主要的磁盘缓存，以此用来减少对磁盘 I/O 的操作。具体来说，就是把磁盘中的数据缓存到内存中，把对磁盘的访问变为对内存的访问。

当一个进程准备读取磁盘上的文件内容时，操作系统会先查看待读取的数据所在的页（page）是否在页缓存（pagecache）中，如果存在（命中）则直接返回数据，从而避免了对物理磁盘的 I/O 操作；如果没有命中，则操作系统会向磁盘发起读取请求并将读取的数据页存入页缓存，之后再将数据返回给进程。

同样，如果一个进程需要将数据写入磁盘，那么操作系统也会检测数据对应的页是否在页缓存中，如果不存在，则会先在页缓存中添加相应的页，最后将数据写入对应的页。被修改过后的页也就变成了脏页，操作系统会在合适的时间把脏页中的数据写入磁盘，以保持数据的一致性。

用过 Java 的人一般都知道两点事实：对象的内存开销非常大，通常会是真实数据大小的几倍甚至更多，空间使用率低下；Java 的垃圾回收会随着堆内数据的增多而变得越来越慢。基于这些因素，使用文件系统并依赖于页缓存的做法明显要优于维护一个进程内缓存或其他结构，至少我们可以省去了一份进程内部的缓存消耗，同时还可以通过结构紧凑的字节码来替代使用对象的方式以节省更多的空间。

此外，即使 Kafka 服务重启，页缓存还是会保持有效，然而进程内的缓存却需要重建。这样也极大地简化了代码逻辑，因为维护页缓存和文件之间的一致性交由操作系统来负责，这样会比进程内维护更加安全有效。

ii、零拷贝

除了消息顺序追加、页缓存等技术，Kafka 还使用零拷贝（Zero-Copy）技术来进一步提升性能。所谓的零拷贝是指将数据直接从磁盘文件复制到网卡设备中，而不需要经由应用程序之手。零拷贝大大提高了应用程序的性能，减少了内核和用户模式之间的上下文切换。对 Linux 操作系统而言，零拷贝技术依赖于底层的 sendfile() 方法实现。对应于 Java 语言，FileChannal.transferTo() 方法的底层实现就是 sendfile() 方法。

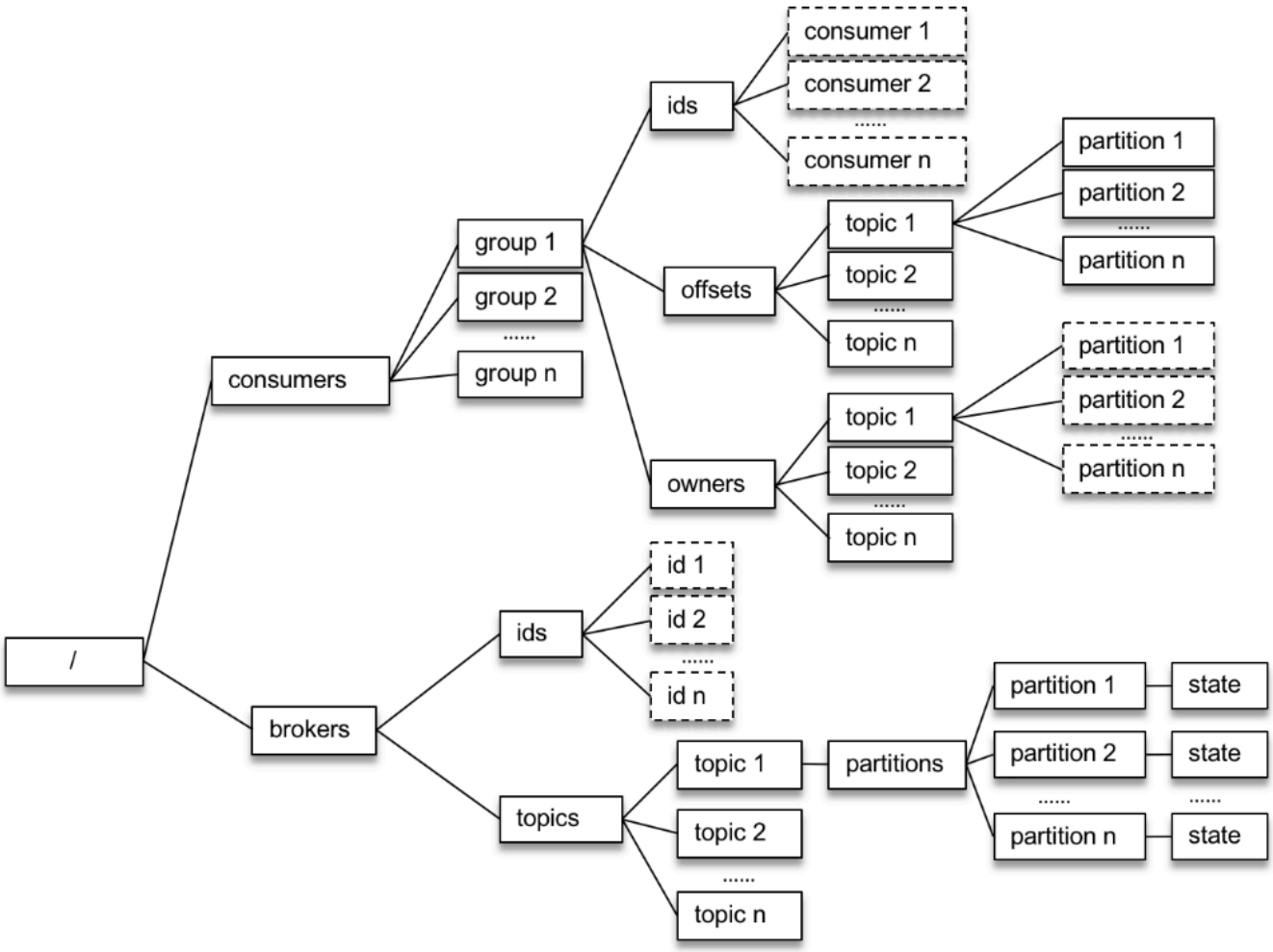
12、聊一聊Kafka的延时操作的原理（了解）

Kafka 中有多种延时操作，比如延时生产，还有延时拉取（DelayedFetch）、延时数据删除（DelayedDeleteRecords）等。
延时操作创建之后会被加入延时操作管理器（DelayedOperationPurgatory）来做专门的处理。延时操作有可能会超时，每个延时操作管理器都会配备一个定时器（SystemTimer）来做超时管理，定时器的底层就是采用时间轮（TimingWheel）实现的。

13、聊一聊Kafka控制器的作用（了解）

在 Kafka 集群中会有一个或多个 broker，其中有一个 broker 会被选举为控制器（Kafka Controller），它负责管理整个集群中所有分区和副本的状态。当某个分区的 leader 副本出现故障时，由控制器负责为该分区选举新的 leader 副本。当检测到某个分区的 ISR 集合发生变化时，由控制器负责通知所有broker更新其元数据信息。当使用 kafka-topics.sh 脚本为某个 topic 增加分区数量时，同样还是由控制器负责分区的重新分配。

14、Kafka的旧版Scala的消费者客户端的设计有什么缺陷？（了解）



如上图，旧版消费者客户端每个消费组在 ZooKeeper 中都维护了一个 /consumers//ids 路径，在此路径下使用临时节点记录隶属于此消费组的消费者的唯一标识（consumerIdString），/consumers//owner 路径下记录了分区和消费者的对应关系，/consumers//offsets 路径下记录了此消费组在分区中对应的消费位移。

每个消费者在启动时都会在 /consumers//ids 和 /brokers/ids 路径上注册一个监听器。

> 当 /consumers//ids 路径下的子节点发生变化时，表示消费组中的消费者发生了变化；
>
> 当 /brokers/ids 路径下的子节点发生变化时，表示 broker 出现了增减。

这样通过 ZooKeeper 所提供的 Watcher，每个消费者就可以监听消费组和 Kafka 集群的状态了。

这种方式下每个消费者对 ZooKeeper 的相关路径分别进行监听，当触发再均衡操作时，一个消费组下的所有消费者会同时进行再均衡操作，而消费者之间并不知道彼此操作的结果，这样可能导致 Kafka 工作在一个不正确的状态。与此同时，这种严重依赖于 ZooKeeper 集群的做法还有两个比较严重的问题。

1. **羊群效应（Herd Effect）**：所谓的羊群效应是指ZooKeeper 中一个被监听的节点变化，大量的 Watcher 通知被发送到客户端，导致在通知期间的其他操作延迟，也有可能发生类似死锁的情况。
2. **脑裂问题（Split Brain）**：消费者进行再均衡操作时每个消费者都与 ZooKeeper 进行通信以判断消费者或broker变化的情况，由于 ZooKeeper 本身的特性，可能导致在同一时刻各个消费者获取的状态不一致，这样会导致异常问题发生。

15、消费再均衡有哪些场景及相应的原理是什么？（理解）

就目前而言，一共有如下几种情形会触发再均衡的操作：

- 有新的消费者加入消费组。

- 有消费者宕机下线。消费者并不一定需要真正下线，例如遇到长时间的GC、网络延迟导致消费者长时间未向 GroupCoordinator 发送心跳等情况时，GroupCoordinator 会认为消费者已经下线。
- 有消费者主动退出消费组（发送 LeaveGroupRequest 请求）。比如客户端调用了 unsubscrible() 方法取消对某些topic的订阅。
- 消费组所对应的 GroupCoorinator 节点发生了变更。
- 消费组内所订阅的任一topic或者topic的分区数量发生变化。

GroupCoordinator 是 Kafka 服务端中用于管理消费组的组件。而消费者客户端中的 ConsumerCoordinator 组件负责与 GroupCoordinator 进行交互。

第一阶段（FIND_COORDINATOR）

消费者需要确定它所属的消费组对应的 GroupCoordinator 所在的 broker，并创建与该 broker 相互通信的网络连接。如果消费者已经保存了与消费组对应的 GroupCoordinator 节点的信息，并且与它之间的网络连接是正常的，那么就可以进入第二阶段。否则，就需要向集群中的某个节点发送 FindCoordinatorRequest 请求来查找对应的 GroupCoordinator，这里的“某个节点”并非是集群中的任意节点，而是负载最小的节点。

第二阶段（JOIN_GROUP）

在成功找到消费组所对应的 GroupCoordinator 之后就进入加入消费组的阶段，在此阶段的消费者会向 GroupCoordinator 发送 JoinGroupRequest 请求，并处理响应。

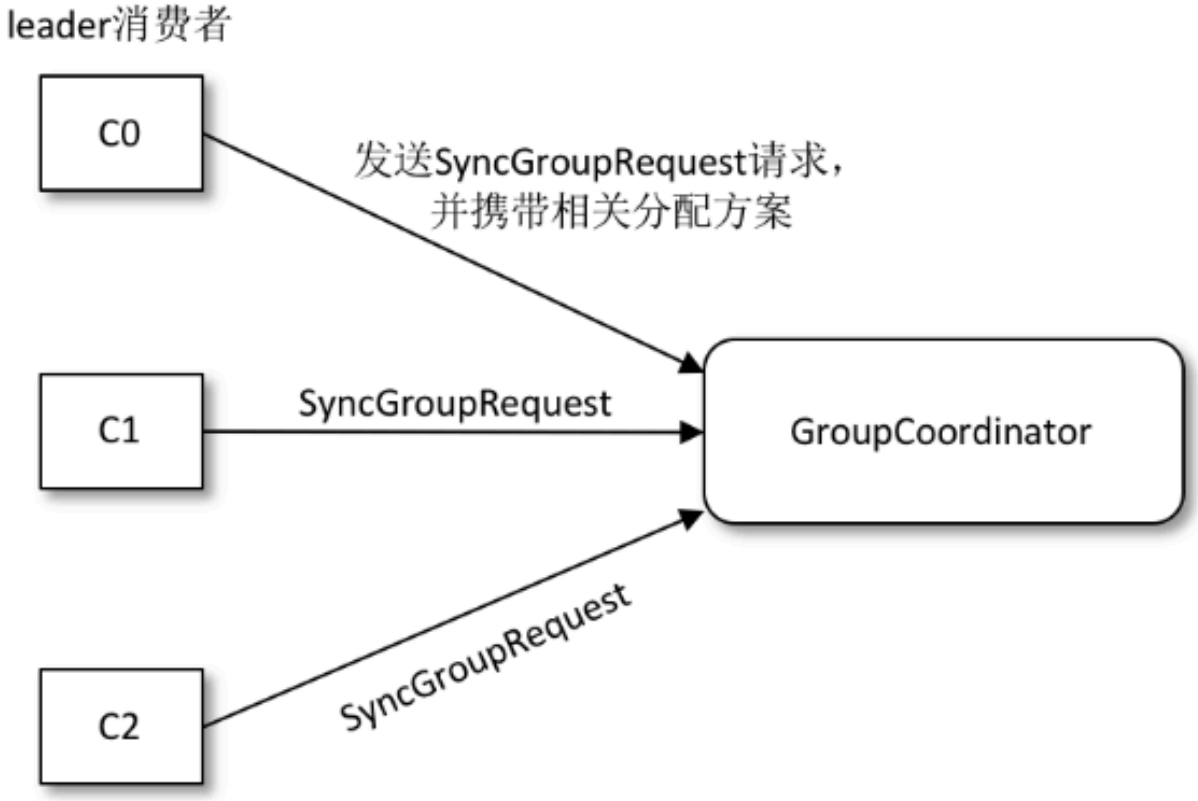
选举消费组的leader
如果消费组内还没有 leader，那么第一个加入消费组的消费者即为消费组的 leader。如果某一时刻 leader 消费者由于某些原因退出了消费组，那么会重新选举一个新的 leader

选举分区分配策略

1. 收集各个消费者支持的所有分配策略，组成候选集 candidates。
2. 每个消费者从候选集 candidates 中找出第一个自身支持的策略，为这个策略投上一票。
3. 计算候选集中各个策略的选票数，选票数最多的策略即为当前消费组的分配策略。

第三阶段（SYNC_GROUP）

leader 消费者根据在第二阶段中选举出来的分区分配策略来实施具体的分区分配，在此之后需要将分配的方案同步给各个消费者，通过 GroupCoordinator 这个“中间人”来负责转发同步分配方案的。



第四阶段（HEARTBEAT）

进入这个阶段之后，消费组中的所有消费者就会处于正常工作状态。在正式消费之前，消费者还需要确定拉取消息的起始位置。假设之前已经将最后的消费位移提交到了 GroupCoordinator，并且 GroupCoordinator 将其保存到了 Kafka 内部的 __consumer_offsets topic 中，此时消费者可以通过 OffsetFetchRequest 请求获取上次提交的消费位移并从此处继续消费。

消费者通过向 GroupCoordinator 发送心跳来维持它们与消费组的从属关系，以及它们对分区的所有权关系。只要消费者以正常的时间间隔发送心跳，就被认为是活跃的，说明它还在读取分区中的消息。心跳线程是一个独立的线程，可以在轮询消息的空档发送心跳。如果消费者停止发送心跳的时间足够长，则整个会话就被判定为过期，GroupCoordinator 也会认为这个消费者已经死亡，就会触发一次再均衡行为。

16、Kafka中的生产幂等是怎么实现的？（理解）

为了实现生产者的幂等性，Kafka 为此引入了 producer id（以下简称 PID）和序列号（sequence number）这两个概念。

每个新的生产者实例在初始化的时候都会被分配一个 PID，这个 PID 对用户而言是完全透明的。对于每个 PID，消息发送到的每一个分区都有对应的序列号，这些序列号从0开始单调递增。生产者每发送一条消息就会将 <PID，分区> 对应的序列号的值加1。

broker 端会在内存中为每一对 <PID，分区> 维护一个序列号。**对于收到的每一条消息，只有当它的序列号的值（SN_new）比 broker 端中维护的对应的序列号的值（SN_old）大1（即 SN_new = SN_old + 1）时，broker 才会接收它。**如果 SN_new< SN_old + 1，那么说明消息被重复写入，broker 可以直接将其丢弃。如果 SN_new> SN_old + 1，那么说明中间有数据尚未写入，出现了乱序，暗示可能有消息丢失，对应的生产者会抛出 OutOfOrderSequenceException，这个异常是一个严重的异常，后续的诸如 send()、beginTransaction()、commitTransaction() 等方法的调用都会抛出 IllegalStateException 的异常。

17、Kafka支持什么语义，怎么实现Exactly Once？（理解背诵）

Kafka支持的**消费语义**有以下几种：

at most once：最多消费一次，消息可能会丢失-------log日志

at least once：至少消费一次，但是会重复消费 例如手动异步提交ow set

exactly once：正好一次，不丢失，不重复

1）至少一次：at-least-once 表示的是关闭offset自动提交功能，消费端在消费数据的时候很可能在commitAync之前，已经保存在数据库，但是这个时候服务器宕机了，从而导致offset不能提交成功。这个时候再次启动消费者的时候，还是会再次写入数据库，也就是至少一次会重复消费，至少不会丢数据。

2）至多一次：at-most-once 表示有可能是0次或者1次，可以选择开启自动提交offset的功能，然后把自动提交offset的时间设置一下，有可能消费者在消费的时间段内就到了自动提交的时间，从而导致了offset已经提交了，但是数据库保存还没进行。下一次再消费的时候就会认为offset已经成功了，直接丢弃消息。就会造成丢数据。

3）仅一次：exactly-once 表示数据仅被消费一次，还是开启自动开启ow set提交的功能，可以开启consumer.seek()方法，相当于自己处理分区和offset，可以在此基础上开启事务，保持原子性，只有数据库保存成功再提交offset，保证两者同时成功。

Exactly Once语义

将服务器的ACK级别设置为-1（acks=-1），可以保证Producer到Server之间不会丢失数据，即At Least Once语义。相对的，将服务器ACK级别设置为0，可以保证生产者每条消息只会被发送一次，即At Most Once语义。At Least Once可以保证数据不丢失，但是不能保证数据不重复；相对的，At Least Once可以保证数据不重复，但是不能保证数据不丢失。

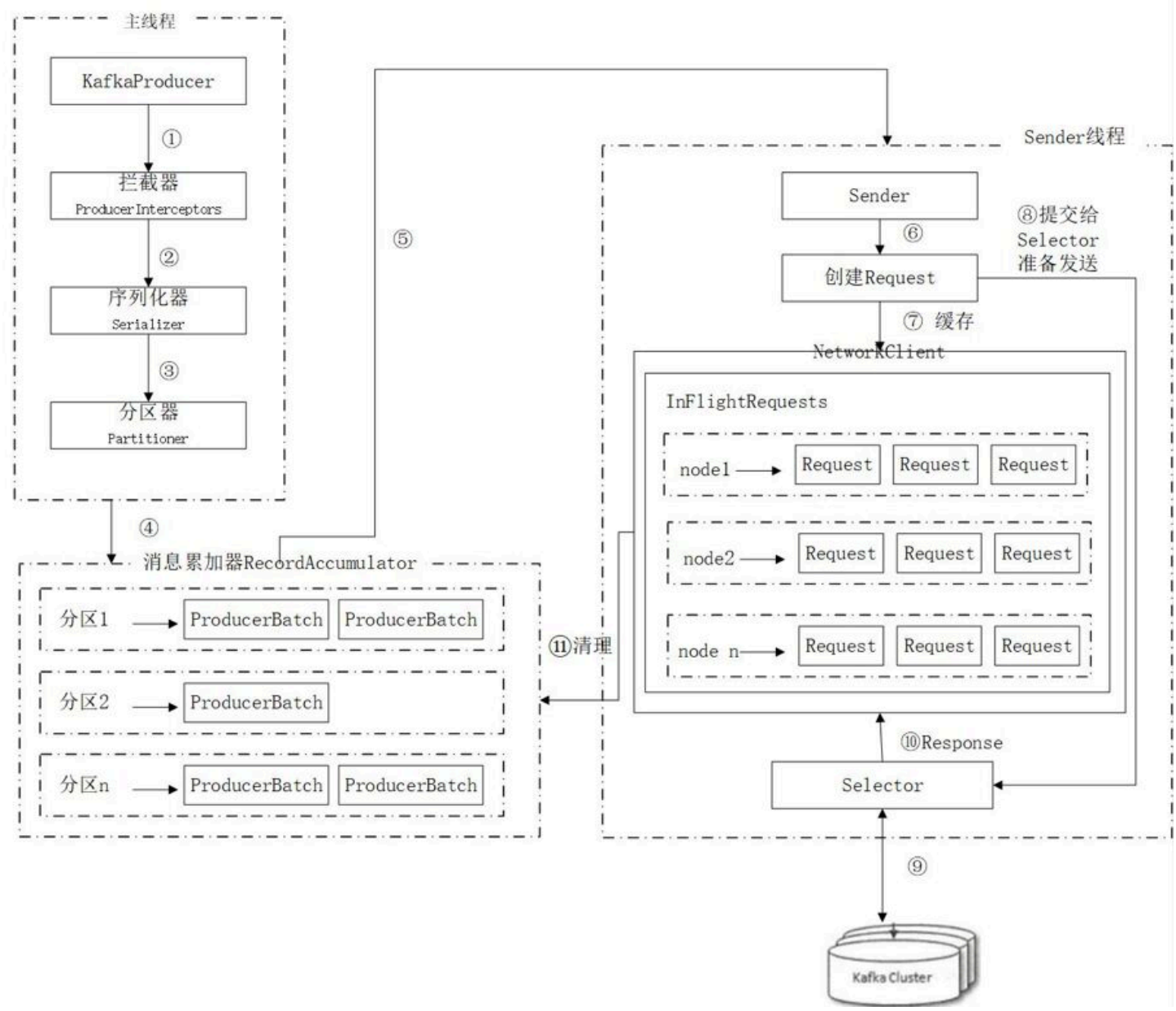
但是，对于一些非常重要的信息，比如说交易数据，下游数据消费者要求数据既不重复也不丢失，即Exactly Once语义。在0.11版本以前的Kafka，对此是无能为力的，只能保证数据不丢失，再在下游消费者对数据做全局去重。对于多个下游应用的情况，每个都需要单独做全局去重，这就对性能造成了很大影响。

0.11版本的Kafka，引入了一项重大特性：**幂等性**。所谓的幂等性就是**指Producer不论向Server发送多少 次重复数据，Server端都只会持久化一条。幂等性结合At Least Once语义，就构成了Kafka的Exactly Once语义**。即：**At Least Once + 幂等性 = Exactly Once**

要启用幂等性，只需要将Producer的参数中**enable.idompotence**设置为true即可。Kafka的幂等性实现其实就是将原来下游需要做的去重放在了数据上游。开启幂等性的Producer在初始化的时候会被分配一个PID，发往同一Partition的消息会附带Sequence Number。而Broker端会对<PID, Partition, SeqNumber>做缓存，当具有相同主键的消息提交时，Broker只会持久化一条。但是PID重启就会变化，同时不同的Partition也具有不同主键，所以幂等性无法保证跨分区跨会话的Exactly Once。

18、Kafka producer的写入数据过程？（背诵）

Kafka的Producer发送消息采用的是异步发送的方式。在消息发送的过程中，涉及到了两个线程——**main线程和Sender线程**，以及一个**线程共享变量——RecordAccumulator**。main线程将消息发送给RecordAccumulator，Sender线程不断从RecordAccumulator中拉取消息发送到Kafka broker。



相关参数

batch.size：只有数据积累到batch.size之后，sender才会发送数据。

linger.ms：如果数据迟迟未达到batch.size，sender等待linger.time之后就会发送数据。

19、Kafka读取消息是推还是拉的模式？有什么好处？（了解）

consumer采用pull（拉）模式从broker中读取数据。

push（推）模式很难适应消费速率不同的消费者，因为消息发送速率是由broker决定的。它的目标是尽可能以最快速度传递消息，但是这样很容易造成consumer来不及处理消息，典型的表现就是拒绝服务以及网络拥塞。

而pull模式则可以根据consumer的消费能力以适当的速率消费消息。pull模式不足之处是，如果kafka没有数据，消费者可能会陷入循环中，一直返回空数据。针对这一点，Kafka的消费者在消费数据时会传入一个时长参数timeout，如果当前没有数据可供消费，consumer会等待一段时间之后再返回，这段时长即为timeout。

20、如何调整生产环境中Kafka主题的分区数量？（理解）

需要注意的是：当我们增加主题的分区数量时，会违背**同一个key进行同一个分区**的事实。我们可以创建一个新的主题，使得该主题有更多的分区数，然后暂停生产者，将旧的主题中的数据复制到新的主题中，然后将消费者和生产者切换到新的主题，操作起来会非常棘手。

21、如何重平衡Kafka集群？（了解）

在下面情况发生时，需要重平衡集群：

- 主题分区在整个集群里的不均衡分布造成了集群负载的不均衡。
- broker离线造成分区不同步。
- 新加入的broker 需要从集群里获得负载。

使用**kafka-reassign-partitions.sh**命令进行重平衡

22、如何查看消费者组是否存在滞后消费？（了解）

我们可以使用**kafka-consumer-groups.sh**命令进行查看，比如：

```
$ bin/kafka-consumer-groups.sh --bootstrap-server cdh02:9092 --describe --group my-group
## 会显示下面的一些指标信息
TOPIC PARTITION CURRENT-OFFSET LOG-END-OFFSET   LAG          CONSUMER-ID HOST CLIENT-ID
主题    分区        当前offset      LEO           滞后消息数       消费者id     主机    客户端id
```

一般情况下，如果运行良好，**CURRENT-OFFSET**的值会与**LOG-END-OFFSET **的值非常接近。通过这个命令可以查看哪个分区的消费出现了滞后。

郑冬综合2502 2026/6/3 21:39:09
涤生大数据版权所有，请勿外传

无标签